A Survey on Machine Learning-Based HPC I/O Analysis and Optimization

Jingxian Peng, Lihua Yang, Huijun Wu, Wenzhe Zhang, Zhenwei Wu, Wei Zhang, Jiaxin Li, Yiqin Dai, and Yong Dong

(Survey Paper)

***Abstract*—The soaring computing power of HPC systems supports numerous large-scale applications, which generate massive data volumes and diverse I/O patterns, leading to severe I/O bottlenecks. Analyzing and optimizing HPC I/O is therefore critical. However, traditional approaches are typically customized and lack the adaptability required to cope with dynamic changes in HPC environments. To address the challenge, Machine Learning (ML) has been increasingly adopted to automate and enhance I/O analysis and optimization. Given sufficient I/O traces from HPC systems, ML can learn underlying I/O behaviors, extract actionable insights, and dynamically adapt to evolving workloads to improve performance. In this survey, we propose a novel taxonomy that aligns HPC I/O problems with learning tasks to systematically review existing studies. Through this taxonomy, we synthesize key findings on research distribution, data preparation, and model selection. Finally, we discuss several directions to advance the effective integration of ML in HPC I/O systems.**

***Index Terms*—High performance computing(HPC), I/O, machine learning(ML), I/O analysis, I/O optimization.**

I. Introduction

As HIGH Performance Computing (HPC) systems rapidly advance in computing power, they can now support large-scale applications that generate petabytes of data in a single execution. The I/O bandwidth of HPC storage systems can easily be saturated by such massive data, and I/O performance can degrade due to contention for shared resources [1], [2]. This results in some applications spending most of their execution time on I/O, making I/O the primary performance bottleneck. Therefore, it is crucial to analyze I/O behavior to identify optimization opportunities and implement targeted improvements for enhancing HPC I/O efficiency.

However, the diversity of I/O behavior and the complexity of storage architectures make it difficult to analyze and optimize I/O. Firstly, HPC I/O workloads no longer only include traditional batch workloads, but also include workloads such as random and metadata-intensive I/O operations [3]. These I/O operations execute simultaneously and contend for shared resources, exacerbating I/O bottlenecks and variability. Secondly, the multi-layer storage and I/O libraries adopted for enhancing performance have complex inter-dependencies. They provide many tunable parameters that need to be coordinated to improve I/O performance.

Many attempts have been made to handcraft statistical equations [4], [5], [6] and simulation models [7], [8] for I/O performance analysis and optimization. These works are better suited for static scenarios because changes in workloads or systems often require manually creating new equations or revising models. Such redesigns and updates are both time-consuming and costly, demanding expertise and experience. Moreover, the vast I/O trace data from HPC systems is challenging to manually distill for valuable insights.

Machine learning (ML), a promising data-driven method, is capable of uncovering latent relationships in large-scale data and prescribing potential actions to optimize I/O. With enough data and appropriate models, ML-based methods can discern patterns and infer knowledge with minimal reliance on prior domain expertise or human intervention. Furthermore, through the continuous acquisition of new data, ML-based methods can adapt to changing workloads. In an era where ML-based I/O analysis and optimization are increasingly prevalent [9], [10], [11], we find it essential to conduct a comprehensive survey of ML methods tailored for HPC I/O. Then, the survey can provide critical insights and guidance for researchers aiming to leverage ML for HPC I/O analysis and optimization.

Several surveys have been conducted on HPC I/O research. Neuwirth et al. [3] investigate parallel I/O evaluation techniques. Bez et al. [12] propose a taxonomy for classifying I/O patterns. Lewis et al. [13] examine ML applications on HPC systems to reveal the impact of their unique I/O patterns. Despite this body of work, the role of ML as a tool for HPC I/O research remains underexplored. To fill this gap, this paper provides a

Received 9 January 2025; revised 17 November 2025; accepted 24 November 2025. Date of publication 3 December 2025; date of current version 30 January 2026. This work was supported in part by the science and technology innovation Program of Hunan Province under Grant 2023RC3021, in part by the National Natural Science Foundation of China under Grant 62302514, Grant 62502531, and Project 202501-YJRC-XX-008, in part by the National Key Laboratory Foundation of China under Grant 2023-KJWPDL-11, and in part by the Natural Science Foundation of Hunan Province of China under Grant 2024JJ6471. Recommended for acceptance by S. Byna. *(Jingxian Peng and Lihua Yang contributed equally to this work.) (Corresponding authors: Wei Zhang; Yiqin Dai.)*

The authors are with the College of Computer Science and Technology, National University of Defense Technology, Changsha 410073, China (e-mail: pjxnudt@nudt.edu.cn; yanglihua@nudt.edu.cn; wuhuijun@nudt.edu.cn; zhangwenzhe@nudt.edu.cn; zhenweiwu@nudt.edu.cn; weizhang@nudt.edu.cn; licyh@nudt.edu.cn; daiyq@nudt.edu.cn; yongdong@nudt.edu.cn).

Digital Object Identifier 10.1109/TPDS.2025.3639682

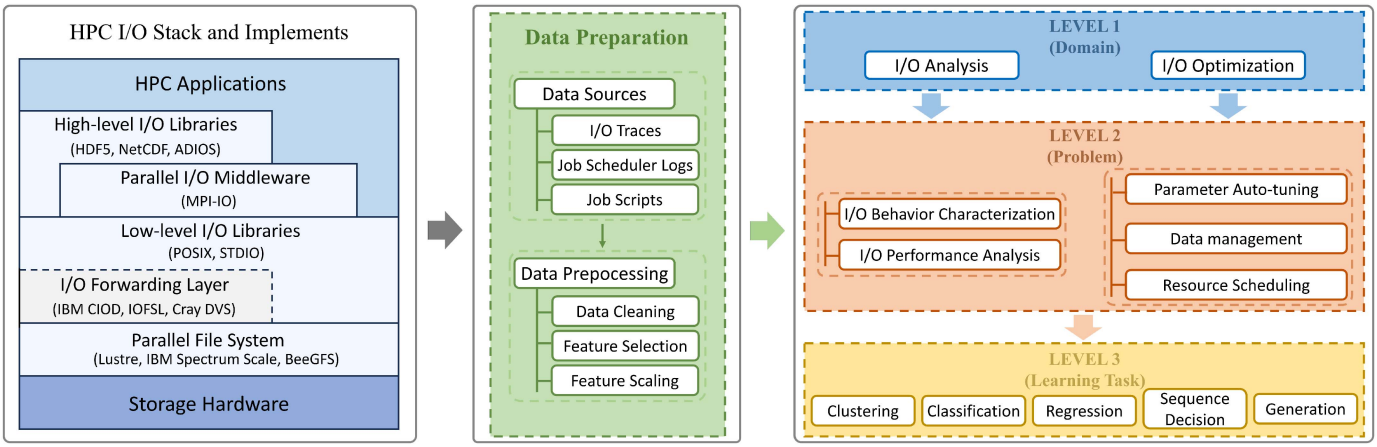


Fig. 1. The process and taxonomy of ML-based HPC I/O analysis and optimization.

systematic review and taxonomy of ML-based methods for HPC I/O analysis and optimization. The main contributions are: (1) We propose a three-level taxonomy that categorizes the application of ML to HPC I/O across domains, problems, and learning tasks, as shown on the right of Fig. 1. This taxonomy provides a structured framework and insights into the connections between these two fields. (2) We summarize data preparation practices in existing studies, including data sources and preprocessing techniques. (3) Based on this taxonomy, we derive insights from three perspectives and discuss future directions to advance ML for HPC I/O.

The rest of this paper is organized as follows. Section II presents the current state of I/O in HPC systems and its intersection with ML. Section III summarizes data sources and preprocessing methods covered in this survey. Sections IV and V survey ML-based approaches to HPC I/O analysis and optimization, respectively, using the proposed taxonomy. Section VI presents findings and insights across research distribution, data preparation, and model selection. In Section VII, we discuss future work to advance ML-based HPC I/O research. Finally, Section VIII concludes the paper.

II. Background

To understand recent advances in ML-based HPC I/O research, this section provides essential background on HPC I/O systems and ML. Section II-A introduces the architecture of a typical HPC I/O system, and Section II-B discusses the primary challenges in analyzing and optimizing HPC I/O in practice. This establishes a foundation for understanding the problems that ML aims to address. Section II-C defines the scope and methodology of this survey. Finally, Section II-D presents an overview of learning tasks and ML techniques used in the HPC I/O domain.

A. HPC I/O System Architecture

A typical HPC I/O system architecture can be abstracted as a Client-Server (CS) architecture, where the two are interconnected via Ethernet or high-speed network, such as InfiniBand (IB). The clients are usually compute nodes (CNs) or dedicated I/O nodes (IONs), which are responsible for initiating I/O requests from applications to the server. The server runs a Parallel File System (PFS), which is responsible for receiving and processing I/O requests, interacting with storage hardware, and returning the results to the clients.

To efficiently manage the flow of data between applications and storage hardware, HPC systems provide a multi-layer I/O software stack, as shown on the left of Fig. 1. It consists of a series of I/O libraries, an I/O forwarding layer, and parallel file systems. High-level I/O libraries (e.g. HDF5 [14], NetCDF [15], ADIOS [16]) offer data descriptions and storage methods that facilitate portability and high performance for scientific computing. Applications can also directly use the MPI-IO middleware library [17] to perform parallel I/O, or use low-level I/O libraries such as POSIX to execute serial I/O. The I/O forwarding layer (e.g., CIOD [18], IOFSL [19], Cray DVS [20]) exists only in HPC systems that use IONs, which is responsible for aggregating and forwarding I/O requests from CNs. This layer alleviates the load on servers and helps CNs decouple I/O operations. Finally, PFS divides files into multiple data blocks and aggregates multiple storage hardware to read and write data in parallel, while providing a global and shared file view to the upper layers. Examples include IBM Spectrum Scale (based on the previous GPFS) [21], Lustre [22] and BeeGFS [23], which support large-scale parallel I/O operations to accommodate the rapid processing of large amounts of data in HPC environments.

B. Challenges in HPC I/O Analysis and Optimization

The multi-layer storage architecture is designed to improve I/O performance and facilitate scientific research. However, the complexity of multi-layer storage architecture, the sharing mechanism of storage system, and the large number of I/O operations present a series of challenges.

Firstly, there is the difficulty of I/O management brought about by the multi-layer storage architecture. Each layer provides adjustable parameters that determine how to move data between intermediate nodes and how to store data on disk to

achieve the high-performance requirements of different workloads. These interactive operations result in complex I/O patterns. Bez et al. [12] conduct a comprehensive survey of I/O access patterns in HPC systems, establishing an important foundation to understand HPC I/O behavior. However, the I/O path is still difficult to model and is a closed box for users and system administrators. The parameters need to coordinate with each other; otherwise, it may lead to performance degradation [24]. The lengthy I/O path, along with a complex software stack and configurations, makes it increasingly challenging for HPC system maintainers and application developers to leverage the I/O performance of the system.

Secondly, fluctuations in I/O performance require continuous adjustments in analysis and optimization to adapt to the current workload and system state. Unlike computational resources that can be used exclusively, the system components of I/O are shared by applications, leading to significant interference in their I/O performance, manifested as I/O variability [2]. The manifestations and sources of the variability are diverse. It may be a temporary impact due to contention, or a performance change resulting from the long-term operation of the storage system (system aging or updating) [25].

Thirdly, the amount of data used for I/O analysis and optimization work is substantial and the work is time-consuming and limited to I/O experts. On large-scale supercomputers, recorded trace logs can exceed 100 GB in a single day [26]. Moreover, researchers need to have sufficient knowledge and experience regarding HPC I/O and data analysis to sort out the interactions between applications and I/O systems, in order to identify the root causes of performance issues.

C. Scope and Survey Methodology

HPC I/O analysis and optimization constitute a widely researched field, as efficient I/O is critical for application performance. However, modern HPC systems (e.g., multi-layer storage) and applications (e.g., various access pattern) render I/O behavior characterization, performance prediction, and optimization extremely complex. Traditional methods often struggle with these intricate and dynamic patterns. While ML offers a promising alternative, current research efforts lack a unified analytical framework.

To bridge this gap, we focus on 82 papers that apply ML to HPC I/O analysis and optimization through a systematic retrieval[1] and rigorous screening.[2] Unlike algorithm-centric surveys, our work categorizes existing studies from an integrative perspective that examines the intersection of I/O problems and ML techniques. This taxonomy reveals core principles, technical routes, and applicable scenarios across a range of ML methods. Ultimately, it provides HPC researchers, system developers, and storage architects with a clear technical roadmap and fosters dialogue between the storage systems and ML research communities.

D. Machine Learning in HPC I/O

ML is widely applied to data analysis because many HPC I/O problems can be naturally cast as ML tasks, including: (1) *Clustering* discovers unknown groups or patterns in data; representative algorithms include k-means and DBSCAN. (2) *Classification* assigns data instances to predefined categories; common classifiers include Decision Tree (DT) and Random Forest (RF). (3) *Regression* predicts continuous values for data instances; typical models include Classification and Regression Tree (CART) and Neural Networks (NNs). (4) *Sequential Decision* involves learning optimal policies to achieve long-term goals through sequential interactions in dynamic environments; it is typically solved using Reinforcement Learning (RL) algorithms, such as Deep Q-Network (DQN) and Deep Deterministic Policy Gradient (DDPG). (5) *Generation* is a task that produces new data instances that approximate the underlying data distribution; Hidden Markov Model (HMM), Variational Autoencoder (VAE), and Large Language Model (LLM) are three approaches adopted in HPC I/O research.

This taxonomy of learning tasks helps clarify primary approaches for specific problems and facilitates cross-domain knowledge transfer by identifying common approaches across different problems.

III. Data Preparation

Data is a critical factor influencing the effectiveness of ML techniques. We summarize data preparation in existing studies, including data sources and data preprocessing.

A. Data Sources

We observe three primary types of data sources: I/O traces, job scheduler logs, and job scripts. I/O traces capture detailed I/O behavior at the application-layer and file system activities at the system-layer. To acquire these traces, researchers execute benchmark tools (e.g., IOR and IOZone) or applications on HPC systems while deploying collection tools. Specialized tools such as Darshan [27], Recorder [28], and LMT [29] are widely used. Darshan is a lightweight tool designed for characterizing long-term I/O workloads primarily at the application-layer. It uses counters to record MPI-IO and POSIX-IO operations and other statistics. Consequently, most datasets covering several months are collected by Darshan [9], [30]. In contrast, LMT monitors file system activities, including metrics such as OSS write throughput [31]. Job scheduler logs come from native logs or databases of schedulers such as Slurm. They record job lifecycle events, user information (e.g., UID), allocated resources (e.g., number of nodes and memory), and job IDs. These records provide crucial context regarding jobs, resource utilization, and system load, which helps to understand the environment of I/O operations (e.g., number of concurrent jobs) [32]. Job scripts are the script files submitted by users. They contain the configuration instructions (e.g., resource requests) and the

[1]Keywords *"HPC OR high performance computing" AND "I/O"* are used to search two databases (i.e., ACM Digital Library and IEEE Xplore) to obtain papers from 2000 to 2025, followed by snowballing. Studies gathered from both methods are subjected to the same inclusion criteria.

[2]Two inclusion criteria: (1) involvement of HPC experimental platforms or datasets, and (2) use of ML to analyze or optimize I/O problems.

execution command sequences (e.g., specifying input and output files). This information can reveal user intent and anticipated I/O patterns [33].

Furthermore, these data sources can be integrated to enable more comprehensive I/O analysis. Both application-layer I/O traces and job scheduler logs contain job IDs, a feature that allows them to be correlated. In contrast, system-layer I/O traces record filesystem activities by timestamp and typically lack explicit job or process identifiers (e.g., job IDs and PIDs). Therefore, studying application I/O behavior using system-layer data requires correlating its timestamps with those from job scheduler logs or application-layer traces to isolate the corresponding activities [34].

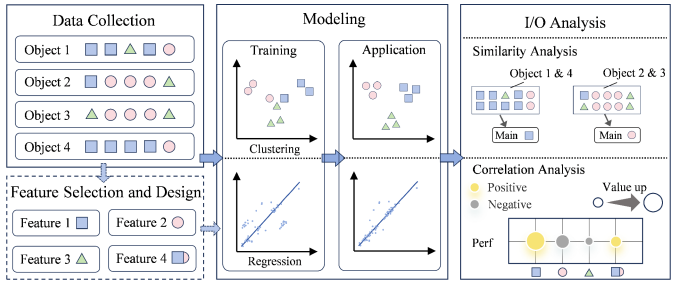


Fig. 2. The framework of clustering-based and regression-based I/O analysis. Objects represent log data and contain different I/O behaviors. Different polygons represent different I/O behaviors.

B. Data Preprocessing

The presence of issues such as missing values, outliers, and skewed distributions in data sources necessitates the use of multiple processing techniques to create a suitable dataset for ML model training. We summarize three primary data preprocessing techniques.

1) Data Cleaning: Data cleaning refers to the process of removing noisy data or imputing missing values. Noisy data typically includes outliers and data irrelevant to the research objectives. For example, jobs with a total I/O volume below 1 GB are excluded to mitigate the impact of insignificant I/O on the model [35]. Essential missing values are imputed using methods such as fixed values [36] or linear interpolation [37].

2) Feature Selection: Feature selection involves identifying features that are both relevant to the objective and informative. In most studies, researchers manually design and extract features based on study objectives and domain knowledge [38], [39], [40]. To reduce redundancy within the feature set, specific algorithms are employed. For example, the Spearman correlation coefficient is used to eliminate redundant features by quantifying their interrelationships [41], while Principal Component Analysis (PCA) identifies key features through dimensionality reduction [42].

Recent research has explored the capability of Convolutional Neural Networks (CNNs) to automate feature extraction, thereby improving efficiency and uncovering complex patterns that are challenging to design manually. For example, the diverse syntax and commands in job scripts make it difficult to develop universal parsers for metric extraction. However, once these scripts are transformed into an image-like representation using techniques like Word2Vec, CNNs can extract features directly to eliminate the need for custom parsers [33], [43]. Similarly, I/O metrics can be organized into grids or image-like structures to facilitate the extraction of more complex features using CNNs [11], [44].

3) Feature Scaling: Feature scaling addresses the performance degradation in ML models caused by large disparities in feature scales or heavily skewed distributions. This process transforms different metrics to a comparable scale or a more stable distribution. For instance, the hundreds of counters used by tools like Darshan to record I/O traces often exhibit vastly different and wide-ranging scales [36]. Consequently, most studies apply logarithmic transformations to improve the numerical distribution within features [30], [45]. They then employ techniques such as normalization to rescale features to [0, 1] range [31], or Z-score standardization to set the mean to 0 and variance to 1 [46].

IV. ML-Based I/O Analysis

ML-based I/O analysis aims to automate and enhance the understanding of I/O through data-driven approaches. Research in this area addresses two core questions: I/O behavior characterization and I/O performance analysis. The former seeks to uncover underlying patterns in I/O activities, while the latter focuses on diagnosing the factors affecting performance. Fig. 2 presents the technical routes of primary learning tasks.

A. I/O Behavior Characterization

1) Clustering: Clustering divides the research objects into several groups, making the objects within each group to share some potential attributes. These attributes serve as representative features of the group, helping to reveal I/O patterns and trends. K-means, valued for its ease of implementation, is employed to cluster jobs to uncover the dominant I/O patterns of HPC workload [9], [35]. Although both studies perform job clustering, their choice of features captures distinct perspectives on I/O patterns, as shown in Table I. Pavan et al. [9] design I/O phases based on I/O operation timestamps to model job behavior. Their analysis of the phase distribution within clusters reveals which phases affect the system and applications, respectively. In contrast, Bang et al. [35] develop a feature selection algorithm to extract log metrics highly correlated with I/O performance. These metrics serve as clustering features to characterize I/O behavior from a resource usage perspective.

However, k-means requires predefining the number of clusters, making it unsuitable for analysis where the true cluster structure is unknown or has a non-intuitive number of clusters. In such cases, density-based clustering [34], [37], [38], [47] or hierarchical clustering [46] is preferred, as they can automatically discover clusters of arbitrary shapes without this prerequisite. For example, the periodic and repetitive I/O bursts in traditional scientific applications enable server-side profiling to reduce characterization overhead. But server-side logs contain mixed I/O traffic from concurrent jobs, introducing significant noise and an unknown number of burst clusters. To address this,

TABLE I
ML-BASED HPC I/O ANALYSIS

Learning Tasks	Input Features	ML Methods	Detailed Algorithms	Reference
I/O Behavior Characterization				
Clustering	I/O burst attributions[1]	Density Clustering	CLIQUE	[37]
			OPTICS	[34]
	Darshan metrics[2]	Distance Clustering	K-means	[35]
	Features derived from Darshan metrics			[9]
		Hierarchical Clustering	HDBSCAN	[46]
	Coding sequence derived from time-series data	Density Clustering	Customized	[47]
	Job runtime, Number of allocated cores		DBSCAN	[38]
Classification	UID, ReqCPUs, SubmitTime, JobPath, JobName	Classification Model	DT	[48]
	Workdir, Jobname, Cluster, Jobid, Gid, Uid, ReqCPUs, ReqNodes, Partition		RF	[49]
	Darshan metrics		DT, RF, XGBoost	[32] [50]
I/O Performance Analysis				
Clustering	Darshan metrics	Hierarchical Clustering	Agglomerative	[51]
	Throughput statistics vector			[52]
	Throughput vector of I/O phase	Density Clustering	DBSCAN	[53]
Classification	Application I/O request states, States of the storage server	Classification Model	NN	[54]
Regression	Darshan metrics	Regression Model	K-means + KNN	[55]
			HDBSCAN + XGBoost	[30]
			Ensemble model	[36]
	Darshan metrics, LMT metrics[3], Scheduler metrics[4]		KNN, GBDT, RF	[31]
	Access size, Access type, Offset, Delta-offset, Features derived from performance error class		NN	[56]
	Number of clients, Number of servers, Array size		RBF NN	[57]
	Numerical vector of multi-timestep I/O configuration parameters from concurrent applications		LSTM	[58]
	Features derived from multi-stage write path		LinearRegression	[39]
			LASSO	[59] [60]
	CPU clock frequency, (VM)I/O scheduling policy, Number of threads, File size, Record size, I/O op mode		MARS, LSP	[40]
			Multiple models	[61]
	Darshan metrics, LMT metrics, Features derived from LMT metrics	Bayesian Learning	GPR	[41]
			CAVE	[62]
	CPU clock frequency, Number of threads, File size, I/O op mode		LMGP	[45]
Generation	Latency states	Ganeration Model	HMM	[63]
	Darshan metrics		LLM	[64] [65]

[1] I/O burst attributions : I/O burst's time, peak value or total volume.
[2] Darshan metrics : The metrics from Darshan log.
[3] LMT metrics : The metrics from LMT log.
[4] Scheduler metrics : The metrics from Scheduler (e.g., Slurm) log.

IOSI [37] uses CLIQUE to identify target bursts. The adjustable grid size of CLIQUE can effectively handle the offset and scaling of bursts caused by I/O contention. AID [34] employs OPTICS for its robustness to noise, identifying multiple I/O burst clusters from server-side logs to extract representative burst for I/O-intensive applications. Furthermore, Yazdani et al. [46] discover varying job densities across users during their analysis of I/O patterns. They employ HDBSCAN, a density-based hierarchical clustering algorithm. It eliminates the need to manually specify a distance threshold, making it capable of discovering clusters with varying densities.

2) Classification: Classification assigns research objects to predefined I/O patterns, thus abstracting complex I/O behaviors into comprehensible information. Yang et al. [48] analyze the distribution of I/O bursts on compute nodes and design spatiotemporal I/O burst metric to quantify the burstiness degree of jobs. Using this metric as a label, they train a decision tree to predict the spatiotemporal burstiness of jobs from metadata available at submission time. The decision tree achieves higher accuracy compared to a grouping strategy that relies on job names. In a similar approach, IOScout [49] uses only job description information to predict I/O characteristics of jobs and emphasizes the importance of working directory in discriminating I/O behaviors. Furthermore, trained models can be used to validate and understand the relationships between features and patterns. Park et al. [32], [50] demonstrate how log metrics can distinguish HPC applications, revealing significantly distinct numerical patterns across applications. They validate the effectiveness of these metrics for application identification using multiple classification models, thus establishing the metric set as a signature for characterizing application I/O behavior.

B. I/O Performance Analysis

Research on I/O performance analysis can be classified into three complementary categories based on primary objectives. The first models performance as a deterministic quantity, aiming to quantify the mapping between various factors and performance metrics. The second models the inherent uncertainty in performance, focusing on analyzing the variability. The third shifts from numerical prediction to automated diagnosis, generating semantic, human-interpretable reports.

1) Deterministic Performance Modeling: This category formulates I/O performance analysis as a *Regression* task, constructing models that learn the functional relationship between I/O workload, system configuration, and a target performance

metric. While a range of regression models is applicable, the reliability of the analysis depends on high model accuracy. We identify three key strategies for enhancing predictive accuracy: feature engineering, local modeling, and application of complex models.

Feature engineering encompasses feature design, feature transformation, and feature selection. In feature design, domain knowledge is encoded in features to capture the causes of performance variation. Schmid et al. [56] observe that even with identical access parameters, I/O access times can vary by orders of magnitude due to differences in I/O paths. To address this, they characterize I/O paths as predictive error classes for use as model features, enabling the model to distinguish among paths. Similarly, Xie et al. [60] encode the multi-stage write path of the system into a feature set and build features for interference to improve model accuracy. Feature transformation aims to capture the nonlinear relationships between features and performance metrics, thus allowing simple linear models to effectively characterize complex system behaviors [39]. Meanwhile, feature selection identifies the most predictive features for performance estimation while eliminating redundant ones. Its primary methodologies include filter, wrapper, and embedded methods. Filter methods calculate the correlation between features and I/O performance before model training, selecting those with higher associations to form the final feature set [31], [55]. Wrapper methods iteratively train models on different feature subsets, aiming to identify the combination that yields the highest accuracy [39]. Embedded methods assign importance weights to features during model training, which signify their contribution to the model. An example is LASSO regression, which employs L1 regularization to constrain the feature set [60].

Local modeling refers to training separate models for local data partitions that share similar distributions within a dataset. This approach enhances predictive accuracy by simplifying the learning task for each local region. Isakov et al. [30] cluster jobs with similar I/O behavior using the HDBSCAN algorithm, which creates a tree structure allowing users to select clusters at different granularities and build performance models for further analysis. Their results demonstrate that these local models yield higher accuracy than global models. This finding aligns with other studies indicating that model accuracy benefits from targeting applications with similar I/O behaviors, as correlations between features and I/O performance vary with changes in these behaviors [31], [55].

Although simple regression or tree models offer interpretable insights into I/O performance, they are often outperformed by complex models (e.g., ensemble learning and neural networks) in intricate HPC system environments [30], [36], [56], [57], [58]. However, this gain in predictive performance comes at the cost of model interpretability. To address this trade-off, model explanation techniques are employed. Isakov et al. [30] utilize eXtreme Gradient Boosting (XGBoost) for I/O throughput modeling, which demonstrates superior performance on tabular data compared to linear regression and decision trees. They subsequently employ SHapley Additive exPlanations (SHAP) to analyze the impact of I/O features on throughput and their interrelationships. In a similar vein, AIIO [36] combines multiple models and leverages both SHAP and Local Interpretable Model-agnostic Explanations (LIME) to generate explanations, integrating these insights to provide a comprehensive analytical framework.

2) Performance Variability Analysis: Clustering: Clustering serves to construct analytical units by grouping objects with similar I/O behaviors [51] or by clustering performance statistics to analyze variability patterns [52], [53]. Within identified job clusters, Costa et al. [51] quantify performance fluctuations using the Coefficient of Variation (CoV) and Z-score, facilitating the analysis of variability causes. In the context of MPI-IO configuration analysis, SCTuner [52] clusters I/O performance vectors across diverse configurations. This clustering reveals significant read performance variability in the experimental environment, while also identifying specific configurations that achieve consistently high write performance. For anomaly detection, Beacon [53] clusters performance vectors of I/O phases to establish a profile of normal fluctuation patterns, and the outliers identified through this process are flagged as abnormal I/O phases.

Classification: Cross-application I/O interference is a key factor driving I/O variability. Egersdoerfer et al. [54] formulate this issue as a classification problem based on real-time system state, thus advancing the quantitative prediction. They design an in-kernel neural network model that learns the nonlinear relationships between application I/O patterns and system load states to predict the degree of performance degradation under specific system environments.

Regression: In contrast to deterministic modeling, this regression task treats I/O performance as a random variable, with the aim of understanding and modeling its statistical variability. Consequently, studies commonly utilize performance variance or distribution to quantify this variability. Bayesian models, which provide not only point predictions (mean) but also a measure of predictive uncertainty (variance), are particularly suited for this purpose [41], [61], [62]. For example, Madireddy et al. [41] employ Gaussian Process Regression (GPR) to predict I/O time, a method that naturally provides statistical information such as standard deviation and covariance. Furthermore, the inverse of the length scale hyperparameter in the covariance function can be interpreted as indicating the relative importance of features, revealing which factors are most influential in driving variability. To improve prediction accuracy, Madireddy et al. [62] employ Conditional Variational Autoencoders (CVAE), which learn the mean and variance parameters by maximizing the probability of the observed data under the reconstructed distribution. To enhance interpretability, MOANA [40] employs regression models to characterize performance variance and builds dedicated models for each I/O pattern. By analyzing the variance distribution across these patterns, it can identify patterns associated with either low or high variability. However, characterizing I/O variability solely through simple statistics is often insufficient, as different underlying distributions can share identical means and variances [66]. Moreover, while variance-based analyses typically assume performance follows a normal distribution to simplify modeling, in practice, performance frequently deviates

from this assumption [40]. To handle non-normal performance distributions, Xu et al. [45] develop a Linear Mixed Gaussian Process (LMGP) model to predict the quantile function. This model achieves superior predictive accuracy by directly adapting to the intrinsic distributional characteristics of the data.

Generation: Wan et al. [63] model I/O latency fluctuations using HMM that attributes these variations to stochastic state transitions, thereby capturing performance dynamics caused by interference in shared resources. However, the effectiveness of model relies on short-term stationarity in system load. It performs well under stable conditions satisfying the Markov assumption, but deteriorates during dramatic load changes.

3) Semantic Performance Diagnosis: This category frames I/O performance analysis as a *Generation* task by using LLMs to analyze raw data, simulate expert reasoning, and generate diagnostic reports. ION [64] demonstrates the feasibility of using LLMs for automated I/O diagnosis. It employs a divide-and-conquer prompting strategy, constructing separate, focused prompts for different I/O issues (e.g., Small I/O and Misaligned I/O) and querying an LLM (e.g., GPT-4) in parallel. As an early proof-of-concept, ION matches the diagnostic capability of heuristic tools like Drishti [67] while providing more nuanced and context-aware justifications. Building on this concept, IOAgent [65] introduces a robust architecture to overcome core limitations of using plain LLMs in complex trace analysis. It comprises three core components: a module-based pre-processor that summarizes I/O traces to fit LLM context windows; a Retrieval-Augmented Generation (RAG) mechanism that integrates domain knowledge to provide grounded diagnoses with references, thus enhancing trustworthiness; and a tree-based merging strategy that structurally combines diagnoses from different modules to minimize hallucinations and ensure a coherent final report.

V. ML-Based I/O Optimization

ML-based HPC I/O optimization research can be categorized into three types based on the primary target. Parameter auto-tuning optimizes configurations within the I/O software stack to shape efficient I/O patterns. Data management orchestrates the spatiotemporal state of data to optimize access paths and unlock the potential of hierarchical storage architectures. Resource scheduling coordinates I/O requests and resource to mitigate contention and balance load.

A. Parameter Auto-Tuning

1) Classification: Kunkel et al. [68] employ a CART model to directly map a given access pattern to a predefined optimal parameter configuration. This strategy avoids evaluating all parameter combinations and enables the extraction of interpretable tuning rules from the trained model. However, its output is restricted to a discrete set of predefined configurations, which precludes the discovery of potentially superior parameter settings between these discrete points. In contrast, DIAL [69] enhances robustness by predicting whether a configuration yields a significant performance gain, thereby eliminating the need for precise throughput prediction.

2) Regression: Regression-based auto-tuning learns a functional mapping from system parameters and workload characteristics to a target performance metric. This approach reduces the need for costly executions of different configurations on a real system. By providing fast performance predictions, the model enables rapid parameter space exploration to efficiently pinpoint high-performance configurations. Various regression models such as RF [10], [70] and NNs [71] are employed for I/O performance prediction.

The effectiveness of this tuning paradigm is highly dependent on the accuracy of the predictive model. To address this, three primary strategies are employed. First, models are periodically retrained with newly collected data to maintain accuracy in dynamic environments [10], [70], [72]. Second, models are strategically selected for specific applications, as the optimal model varies with application characteristics [11]. Third, domain-specific knowledge is integrated to offer more robust modeling schemes. Isaila et al. [73] establish analytical models for network and storage operations in collective I/O and subsequently select suitable ML models for specific components within this analytical framework. This hybrid design confines the impact of noise, as any data-driven inaccuracies are localized to the ML components.

Furthermore, real-time parameter tuning requires obtaining model input with low latency during application execution. A2FL [11] investigates the construction of images from I/O metrics to represent I/O patterns as model inputs. Compared to traditional encoding-based methods, this image-based approach enables significantly faster data processing while preserving critical I/O information.

3) Sequential Decision: In this formulation, parameters are dynamically tuned via continuous interaction with the storage system. The decision process involves monitoring system states and adjusting parameters according to the current policy to obtain performance feedback for policy updates that maximize the objective. Bayesian Optimization (BO) [74], [75], [76] and RL [77], [78], [79], [80], [81] are used to perform this process.

In sequential decision using BO, research aims to improve tuning efficiency through three primary strategies: pruning the parameter space, reducing evaluation costs, and optimizing the algorithm. Pruning the parameter space involves identifying key parameters [76] and constraining their value ranges [75]. OPRAEL [76] trains a model to predict I/O bandwidth and then employs SHAP and PFI to identify the subset of parameters with the most significant impact. To constrain the value range, VAE-ABO [75] uses VAE to learn the joint probability distribution of high-performance configurations in a low-dimensional parameter space. This learned distribution enables BO to focus its sampling on the most promising regions of the high-dimensional space, thus accelerating the tuning. Reducing evaluation costs is achieved by building performance models. Both Agarwal et al. [74] and OPRAEL use XGBoost to establish such models, which replace the need for actual application execution with each configuration and thus significantly reduce the overall tuning time. Optimizing the algorithm includes the integration of different algorithms. To overcome the limitations of any single algorithm, OPRAEL integrates Genetic Algorithm (GA), Tree-structured Parzen Estimator (TPE), and BO using a Bagging

approach. This integration enables information exchange among algorithms, resulting in improved tuning efficiency and stability.

In sequential decision using RL, three components are fundamental: the reward function, the action space, and the RL algorithm. The reward function evaluates the outcome of the action taken by the policy in a given system state, thereby guiding the tuning process. It is typically derived from I/O performance metrics, such as throughput differences [77], [78], [80] and relative changes [79], [81]. To mitigate the impact of workload variations and noise, Wentao et al. [78] define the reward as the change in throughput over a fixed-step window. In contrast, ADSTS [80] continuously monitors performance until it stabilizes to compute the reward. The action space defines the feasible region for parameter adjustments. Depending on the action space (i.e., discrete or continuous) supported by the RL algorithm, continuous parameters are discretized using predefined step sizes [77], [78], while discrete parameters are handled by mapping continuous actions to valid values [79]. To alleviate convergence challenges arising from high-dimensional action spaces, ADSTS uses the LASSO to identify important I/O parameters for tuning. The RL algorithm specifies the policy update rule. Recent work increasingly adopts more advanced algorithms. The shift from discrete-action methods (e.g., DQN) to continuous-action approaches (e.g., DDPG) avoids performance loss caused by discretization. Enhanced variants such as Twin Delayed DDPG (TD3) further improve training stability and convergence through algorithmic refinements including delayed policy updates. Furthermore, parallel training and fine-tuning of pre-trained models are used to reduce training overhead [80], [81].

Separately, TunIO [42] employs RL to dynamically adjust the number of iterations in GA and to identify high-impact parameters at each tuning step. This hybrid approach enables auto-tuning to escape performance plateaus and achieve a better trade-off between cost and benefit.

B. Data Management

1) Clustering: Zhou et al. [82] propose a prefetching strategy based on the I/O similarity among data objects. Their approach first clusters objects using DBSCAN and then prefetches those within the same cluster into faster storage tiers to match current access patterns.

2) Classification: By modeling I/O access pattern prediction as a classification task, the system can identify future data demands and perform efficient data prefetching and caching. Oly et al [83]. regard the I/O access sequence as a Markov process and construct a Markov model based on the transition frequencies between file blocks. This simple yet effective design is suitable for predicting repetitive access patterns on a fixed set of files. Madhyastha et al. [84] investigate the use of Artificial Neural Networks (ANNs) and HMMs for classifying file access patterns. While the inherent flexibility of ANNs enables dynamic categorization into known patterns, it also leads to inaccuracies on unknown or irregular inputs, which are misclassified as the most similar known class. In contrast, HMMs achieve higher precision by modeling probabilistic sequences from historical access traces, effectively capturing irregular but recurring behaviors. Another probabilistic model for sequence prediction is the n-gram model, which estimates the conditional probability of the next item given the previous $n-1$ items via maximum likelihood estimation and predicts the next access by choosing the most probable item [85], [86]. Stacker [85] employs n-grams to predict upcoming read requests and prefetches the corresponding data from SSDs to DRAM on staging nodes, thus reducing the access latency perceived by applications. To rapidly adapt to dynamic access patterns, Stacker conducts cascaded searches across n-gram models of multiple orders. Wan et al. [87] model the access patterns of each data object using Markov chains and estimate long-term access probabilities via the stationary distribution, thus guiding data placement onto high-performance storage tiers.

Data placement is also formulated as a classification task in which storage tiers are treated as classes to maximize I/O performance through optimal data-tier assignment. ASL [88] parses scientific workflow description files to extract data features, such as the minimum distance between files and its consumers, to capture data lifecycle context. This enables I/O optimization across the entire workflow rather than for individual I/O requests. In contrast, Xu et al. [89] derive features from I/O traces to characterize the access patterns of each data object. Their approach learns mappings between specific patterns and optimal storage tiers; for example, small random writes are better suited for burst buffers. This approach enables more generalizable data placement across diverse workloads. Furthermore, AIO [90] combines real-time server-side load metrics with I/O access features to incorporate system-wide load awareness into data placement decisions.

3) Regression: Beyond predicting I/O access patterns, engineered data attributes (e.g., access frequency) provide crucial signals for efficient data management. File lifetime is a key factor in eviction decisions. Monjalet et al. [91] define file lifetime as the duration from creation to last access and encode file paths as input features for CNNs to predict this lifetime. To avoid performance penalties from premature eviction caused by underestimation, they employ a quantile loss function that biases the model toward overestimation. Inspired by multi-task learning, Thomas et al. [92] customize the loss function as a weighted average of prediction error and underestimation penalty to maintain high estimation accuracy while controlling underestimation. This lifetime-based eviction strategy only monitors file creation and manages as little metadata as FIFO, but achieves efficiency comparable to LRU by incorporating file attributes [93]. Another strategy, F-LRU [94], defines the sequence of POSIX operations on a file as its lifecycle and uses KNN to predict the number of I/O requests for eviction guidance. However, relying only on individual file history limits the model's ability to capture shared patterns. To address this, Khelili et al. [95] introduce a new distance metric for KNN that incorporates filename similarity, enabling cross-file learning and improving prediction accuracy and efficiency.

To optimize data placement in heterogeneous storage, SIREN [96] trains a CART model for each storage device to capture the relationship between data access patterns and service times. This allows selecting the device with the lowest predicted latency as the target location.

4) Sequential Decision: WorkflowRL [97] addresses the data placement problem in scientific workflows by modeling it as a sequential decision task and using a DQN to minimize the overall I/O time. Under limited high-performance storage capacity, its adaptive placement strategy mitigates resource contention caused by data migration.

C. Resource Scheduling

1) Clustering: By grouping complex I/O behaviors into representative patterns, clustering provides a foundational knowledge base and reduces the complexity of scheduling problems. Yang et al. [98] employ DBSCAN to cluster I/O burst intervals and build an interval pattern library. This library enables the prediction of burst distributions for new jobs based on their assigned clusters, thereby guiding staggered job scheduling to prevent storage system overload. Separately, Zha et al. [99] propose a probabilistic method to predict congestion in burst buffers for I/O request reordering. To address inaccuracies caused by varying I/O phase durations, they use k-means to cluster applications by phase duration and construct dedicated models per cluster, thereby ensuring the consistency required for reliable predictions. To enhance scheduling efficiency, LuxIO [100] employs a dual-clustering strategy. It creates resource templates by applying k-means to storage performance metrics and infers application resource demands via I/O pattern clustering. A submitted application is assigned to a pattern cluster, enabling the reuse of the matched resource template and streamlining the scheduling process.

2) Classification: Classification transforms continuous, multidimensional monitoring data into discrete categories with clear semantics that inform scheduling decisions. Existing studies fall into three categories according to their objectives. The first achieves I/O-aware job scheduling by predicting job attributes or system states. SchedP [43] employs a CNN to predict whether a job will cause I/O bursts on each Object Storage Server (OSS). This sparse (mostly zero) classification output yields highly accurate predictions, allowing the scheduler to combine them with real-time system I/O load and adjust job priorities to prevent I/O contention. AI4IO [101] is a framework comprising PRIONN [33] and CanarIO [102]. PRIONN prevents I/O contention by predicting job runtime and I/O bandwidth. To leverage the strong classification capability of NNs, it transforms the regression task into a multi-class classification task by binning runtime into equal-width intervals (i.e., categories). CanarIO predicts performance degradation in a parallel file system and identifies I/O-sensitive jobs, enabling the scheduler to prioritize non-sensitive jobs during I/O bottlenecks and avoid wasting compute resources. Additionally, Saeedizade et al. [103] predict system-level I/O burst periods and calculate delayed job start times to minimize the overlap of job execution with these bursts.

The second aims to optimize storage resource allocation and balance I/O load. Wan et al. [104] leverage the observed bimodal distribution of I/O performance to classify each OSS's status as either busy or idle, and then direct new I/O requests to OSSs predicted as idle using classifiers. Similarly, Paul et al. [105] discretize the size of I/O write requests into a finite set of states and train a Markov chain to predict the next state, which is mapped to a specific write size for the Object Storage Target (OST) allocation algorithm. To mitigate contention between foreground and background tasks, RISE [106] uses an n-gram model to predict time intervals of foreground data access, thus determining both the timing and volume of background data migrations. Focusing on file striping, Xian et al. [107] predict whether a job is suitable for file striping, minimizing performance interference caused by improper striping. AIOT [108] first employs clustering to derive representative I/O patterns, which serve as labels for a sequence classification model that predicts the pattern of incoming jobs. To handle both sparse and dense sequences effectively, AIOT introduces a self-attention mechanism to capture short- and long-term dependencies in historical pattern sequences. The predicted pattern serves as a key input to heuristic allocation algorithms.

The third automatically adapts I/O scheduling algorithms to dynamic I/O patterns. Bez et al. [109] identify access patterns at the I/O forwarding layer and adjust the time window parameter of the TWINS scheduling algorithm to better match the current workload, thus improving I/O performance. Pang et al. [44] convert I/O request sequences into a grid format and use a CNN to extract spatiotemporal access features. These features are fed into a fully connected neural network to predict the access pattern, which guides the selection of an appropriate scheduling algorithm. Furthermore, AGIOS [110] employs a decision tree that directly maps access patterns and storage device characteristics to the optimal scheduling algorithm, enabling automatic policy selection.

3) Regression: Regression enables fine-grained scheduling by providing precise predictions of continuous variables (e.g., throughput). U-Shape [111] decomposes user-defined execution time goals into per-window throughput targets and trains a CART regression model to predict these targets, which are used to dynamically adjust service window sizes.

4) Sequential Decision: The tuning of scheduling parameters is formulated as a sequential decision process to dynamically adapt to varying I/O workloads. Bez et al. [112] model the tuning of the time window in the I/O forwarding layer as a contextual multi-armed bandit, which can be viewed as a simplified Markov Decision Process (MDP). They employ the lightweight ε-greedy algorithm, whose rapid inference and low overhead make it suitable for the latency-sensitive forwarding layer. However, such simple approaches are inadequate in complex environments. To address system-level I/O congestion control, AIOC^2 [113] coordinates parameters on both client and server sides. As tabular RL algorithms (e.g., Q-learning) struggle with the large state space of Lustre clusters, AIOC^2 employs DQN, which uses a neural network to approximate the state-action value function.

VI. Findings and Insights

We consolidate scattered practical experiences by analyzing research distribution, data preparation, and model selection to extract instructive findings and insights.

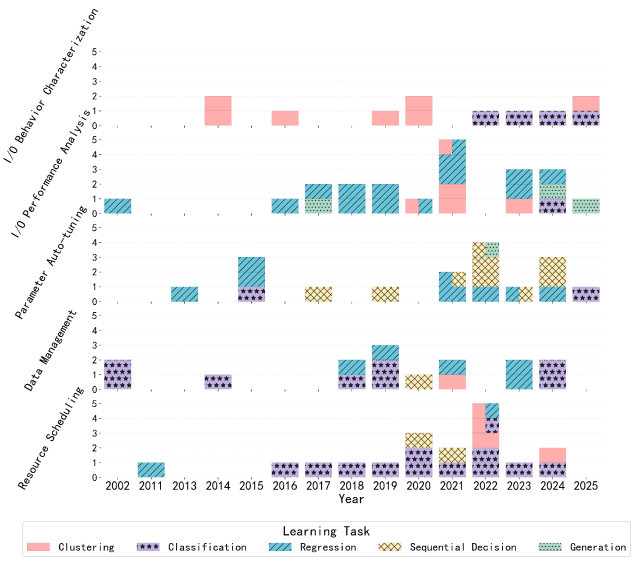


Fig. 3. The evolution of learning tasks across problem fields in ML-based HPC I/O research over time. Each rectangle represents one study, with horizontally divided colored segments indicating multiple learning tasks.

A. Analysis of Research Distribution

We present Fig. 3 to help analyze the distribution and trends of problem fields, as well as the preferences and shifts in learning tasks.

1) Trends in Problem Fields: In the analysis domain, I/O performance analysis remains a mainstream research direction because its objectives are well-defined and easily formulated as regression tasks. Meanwhile, given the high sensitivity of HPC systems to performance, performance prediction and variability analysis continue to be key points. With advances in model interpretation techniques, performance analysis can achieve high accuracy while enabling quantitative causal analysis, which enhances the reliability and insight of results.

Compared to the analysis domain, research in optimization is growing more rapidly. This trend is primarily driven by the increasing complexity of HPC systems where I/O has become a critical performance bottleneck. Various technologies have been developed (e.g., I/O middleware, scheduling policies, and burst buffers), but their configuration introduces significant complexity. Meanwhile, preventing the waste of other resources (e.g., computation) due to I/O bottlenecks motivates I/O-aware co-optimization. These challenges collectively drive the pursuit of automated and intelligent optimization methods.

2) Preferences in Learning Tasks: Clustering serves to characterize I/O behavior and is being adopted in optimization to manage HPC system diversity, thus reducing optimization complexity. For example, clustering system resources into scheduling templates accelerates resource matching [100].

Classification is widely applied in I/O pattern recognition and access sequence prediction to provide well-defined decisions. A common practice involves discretizing continuous variables (e.g., I/O request sizes) to convert regression into classification tasks. Although this simplification enhances robustness, it sacrifices numerical precision.

Regression is commonly employed for precise numerical prediction of performance metrics (e.g., I/O throughput), supporting quantitative analysis and fine-grained optimization.

Sequence decision enabled by RL is increasingly applied in optimization, particularly for long-term optimization in dynamic environments, such as parameter auto-tuning.

Generation tasks are in the exploratory stage, primarily leveraging LLMs to produce interpretable diagnostic reports, enhancing the automation and interactivity of analysis systems.

B. Analysis of Data Preparation

1) Insufficient Dataset: Few studies utilize publicly available datasets (e.g., ALCF I/O Data Repository[3]), which are typically anonymized. Over 90% of data sources are self-collected by researchers and rarely publicly released. This is because HPC I/O behavior is highly dependent on specific system architectures and application workloads, prompting researchers to collect customized data from their own clusters. Moreover, the sensitive nature of HPC I/O data (e.g., user information and system configurations) impedes open sharing. These factors limit reproducibility and cross-study comparisons, hindering the evaluation of model generalization and community collaboration.

Regarding data types, while numerical metrics (e.g., operation counts) dominate the datasets, recent research has begun to incorporate textual data (e.g., file paths) to enhance model performance. This diversification in data types reflects the complexity of HPC I/O behavior and indicates that traditional numerical metrics are insufficient to capture underlying semantic patterns, such as user intent.

2) Word Embedding Techniques: The growing use of textual data has popularized embedding techniques such as Word2Vec. These methods convert text into embeddings that serve as model inputs to capture semantic information.

3) Manual Feature Engineering: Most studies construct features based on HPC I/O knowledge and expert experience. This systematic encoding of domain knowledge into features not only supports hypothesis validation but also enhances model performance. Moreover, manually crafted features are generally interpretable. Furthermore, ML models (except NNs) typically rely on carefully engineered input features. Techniques such as Pearson correlation analysis are commonly used for feature selection to identify key features.

4) Automatic Feature Engineering: NNs can automatically learn features from raw data to reduce the dependence on manual feature engineering and generate richer features. However, the resulting features often lack interpretability. Notably, some studies using NNs still rely on manual feature engineering [44], [56], or only use NNs as feature generators whose output is used as input to non-NN ML models [11]. This suggests that in the HPC I/O domain, learning features exclusively from raw data may not provide enough information to achieve the goal. One reason is that the raw data may lack critical domain-specific knowledge. Another is that available training data may be inadequate for NNs to achieve desired performance.

[3]https://www.mcs.anl.gov/research/projects/darshan/data/

C. Analysis of Model Selection

To inform model selection in ML-based HPC I/O research, we analyze the reasons stated in existing studies and identify three primary factors.

1) Better Performance: This is the most direct reason for model selection, typically drawing on prior research in similar tasks or through comparative evaluation. We find that this reason occurs more frequently in the analysis domain (59%) than in optimization (46%). This discrepancy reflects their distinct objectives: analysis requires precise characterization through high accuracy, while optimization prioritizes effective decisions evaluated by system-level gains (e.g., throughput). However, over-reliance on performance metrics may have negative effects, such as excessive computational overhead.

2) Problem-Model Fit: Most studies first analyze the objectives and data characteristics of problems to match models with appropriate capabilities. We identify four characteristics:

- Cluster Shape and Noise: Most studies perform data cleaning and feature scaling, indicating that HPC I/O data are typically noisy and non-stationary. Thus, in I/O characterization, density-based clustering algorithms (e.g., DBSCAN) are preferred for *operating without predefined cluster numbers*, *robustness to noise*, and *discovery of irregular shapes*.
- Complex Nonlinear Relationships: When I/O performance exhibits complex nonlinear dependencies on parameters, models capturing such patterns are prioritized. NNs are selected as *universal function approximators*, while tree models *inherently handle nonlinear patterns*. In practice, ensemble methods (e.g., RF) are frequently adopted to *enhance generalization and robustness*.
- Uncertainty: For problems requiring uncertainty quantification (e.g., performance variability analysis), Bayesian methods like GPR are preferred because they *provide predictive distributions (e.g., confidence intervals)*.
- Temporal Dependencies: With time-series data, Markov chains and n-grams predict *discrete states and patterns* through sequence probability modeling, while LSTM captures dependencies in *continuous numerical sequences*.

3) Practical Trade-Offs: Model selection in HPC production environments is influenced by three key constraints:

- Interpretability Requirements: In scenarios requiring attribution analysis or expert trust (e.g., performance analysis and parameter auto-tuning), simpler models are preferred. CART are chosen for their *transparent decision rules* [68], while linear models are employed for their *mathematically explainable parameters* [39].
- Implementation Complexity: Algorithms such as DTs and k-means are chosen for their *straightforward implementation* [9]. To avoid complex manual feature engineering, NNs that can *automatically extract features* present an effective solution.
- Runtime Overhead: In practice, some studies select CART over RF for *lower computational cost* [96], or use n-gram models with *negligible training and inference overhead*. In dynamic decision-making, *simplified frameworks* such as contextual bandits are preferred over full MDP modeling to reduce cost [42], [112].

VII. Discussions on Future Trends

Current ML methods in HPC I/O can be enhanced to yield deeper I/O insights. Several research directions are listed to better align ML with the needs of HPC I/O systems.

The scarcity of high-quality, shareable datasets remains a primary bottleneck for applying ML to HPC I/O. The high cost of production runs and data privacy concerns limit data availability, while the labeling required for supervised learning is expensive. Future efforts should focus on constructing shareable I/O datasets through methods such as generative models for trace anonymization [114] and collaborative I/O trace archives [115]. Extracting information from pre-execution data (e.g., job path) offers a promising way to reduce collection costs. Transfer learning lowers the amount of training data needed [116]. Multi-system datasets would enable a more comprehensive understanding of HPC I/O by supporting cross-system model development and testing.

Feature engineering for HPC I/O data should evolve to minimize information loss. Current practices often discard or simplify data that is difficult to process, which may sacrifice valuable insights. An emerging approach that converts logs into images for convolutional feature extraction offers a more informative alternative to traditional methods. However, such conversions must preserve information critical to I/O behavior. For example, when mapping I/O request sequences to images, the adjacency of pixels should reflect the temporal or spatial relationships among requests. Thus, designing effective mappings requires domain knowledge, where comprehensive studies of I/O patterns can provide a theoretical basis [12].

Improving model performance requires a deeper understanding of model errors and interpretability. While ensemble methods that leverage diverse models are increasingly adopted, the key lies in analyzing the sources of modeling errors. Most of the studies surveyed attribute errors to insufficient data and system noise. Isakov et al. [117] classify HPC I/O ML modeling error sources into five types, while other research confirms that performance depends on factors beyond model complexity [118], [118]. This analysis informs both ML method selection and insights into HPC I/O. Furthermore, interpretability techniques remain limited to revealing feature contributions or statistical associations. Thus, the scope of insights a model can provide is constrained by its input [120]. Future work should integrate broader system metrics and explore causal inference methods to uncover root causes.

Lastly, the rise of new workloads necessitates the co-design of storage and network systems. New workloads such as ML applications exhibit I/O patterns (e.g., intensive small-file access) distinct from those of traditional HPC applications [13]. Such patterns result in numerous network requests that intensify network contention. Since network topology and routing algorithms significantly impact I/O performance, it is essential to model the interaction between network parameters (e.g., hop count) and I/O behavior [121], [122]. ML offers a promising

approach to this modeling, enabling cross-layer optimization that could substantially improve I/O performance.

VIII. Conclusion

Given the scale and complexity of modern HPC systems, data-driven ML has emerged as an essential approach for I/O analysis and optimization. In this survey, we systematize commonly used data sources and preprocessing methodologies. Central to our contribution is a novel taxonomy that maps HPC I/O problems to learning tasks, providing a framework through which to interpret and compare existing research. By synthesizing findings across research distributions, data preparation, and model selection, we not only highlight the progress made but also identify key gaps and promising directions. We hope this work serves as a foundation for developing more robust, interpretable, and deployable ML solutions that effectively meet the evolving demands of HPC I/O systems.

References

[1] X. Liang et al., "Improving performance of data dumping with lossy compression for scientific simulation," in *Proc. IEEE Int. Conf. Cluster Comput.*, 2019, pp. 1–11.

[2] E. Karrels et al., "Fine-grained policy-driven I/O sharing for burst buffers," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2023, pp. 1–12.

[3] S. Neuwirth and A. K. Paul, "Parallel I/O evaluation techniques and emerging HPC workloads: A perspective," in *Proc. IEEE Int. Conf. Cluster Comput.*, 2021, pp. 671–679.

[4] M. Dantas, D. Leitão, C. Correia, R. Macedo, W. Xu, and J. Paulo, "Monarch: Hierarchical storage management for deep learning frameworks," in *Proc. IEEE Int. Conf. Cluster Comput.*, 2021, pp. 657–663.

[5] H. Devarajan, A. Kougkas, and X.-H. Sun, "HFetch: Hierarchical data prefetching for scientific workflows in multi-tiered storage environments," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2020, pp. 62–72.

[6] T. Jin, F. Zhang, Q. Sun, M. Romanus, H. Bui, and M. Parashar, "Towards autonomic data management for staging-based coupled scientific workflows," *J. Parallel Distrib. Comput.*, vol. 146, pp. 35–51, 2020.

[7] S. Snyder et al., "Techniques for modeling large-scale HPC I/O workloads," in *Proc. 6th Int. Workshop Perform. Model., Benchmarking, Simul. High Perform. Comput. Syst.*, 2015, pp. 1–11.

[8] X. Luo et al., "ScalaIOExtrap: Elastic I/O tracing and extrapolation," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2017, pp. 585–594.

[9] P. J. Pavan et al., "An unsupervised learning approach for I/O behavior characterization," in *Proc. 31st Int. Symp. Comput. Archit. High Perform. Comput.*, 2019, pp. 33–40.

[10] A. Bağbaba et al., "Improving the I/O performance of applications with predictive modeling based auto-tuning," in *Proc. Int. Conf. Eng. Emerg. Technol.*, 2021, pp. 1–6.

[11] D. K. Sung et al., "A2FL: Autonomous and adaptive file layout in HPC through real-time access pattern analysis," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2024, pp. 506–518.

[12] J. L. Bez, S. Byna, and S. Ibrahim, "I/O access patterns in HPC applications: A 360-degree survey," *ACM Comput. Surv.*, vol. 56, no. 2, pp. 1–41, Sep. 2023.

[13] N. Lewis, J. L. Bez, and S. Byna, "I/O in machine learning applications on HPC systems: A 360-degree survey," *ACM Comput. Surv.*, vol. 57, no. 10, pp. 1–41, May 2025.

[14] M. Folk, G. Heber, Q. Koziol, E. Pourmal, and D. Robinson, "An overview of the HDF5 technology suite and its applications," in *Proc. EDBT/ICDT 2011 Workshop Array Databases*, New York, NY, USA, 2011, pp. 36–47.

[15] C. Lee, M. Yang, and R. A. Aydt, "NetCDF-4 performance report," 2008. [Online]. Available: https://api.semanticscholar.org/CorpusID:15588867

[16] W. F. Godoy et al., "ADIOS 2: The adaptable input output system. A framework for high-performance data management," *SoftwareX*, vol. 12, 2020, Art. no. 100561.

[17] P. Corbett et al., "Overview of the MPI-IO parallel I/O interface," in *Input/Output in Parallel and Distributed Computer Systems*. Boston, MA, USA: Springer, 1996, pp. 127–146.

[18] J. Moreira et al., "Designing a highly-scalable operating system: The blue gene/l story," in *Proc. ACM/IEEE Conf. Supercomput.*, 2006, pp. 53–53.

[19] J. Cope, K. Iskra, D. Kimpe, and R. B. Ross, "Bridging HPC and grid file I/O with IOFSL," in *Proc. Workshop Appl. Parallel Comput.*, 2010, pp. 215–225.

[20] S. Sugiyama and D. Wallace, "Cray DVS: Data virtualization service," 2008. [Online]. Available: https://support.hpe.com/hpesc/public/docDisplay?docId=a00115089en_us&docLocale=en_US

[21] F. Schmuck and R. Haskin, "GPFS: A shared-disk file system for large computing clusters," in *Proc. USENIX Conf. File Storage Technol.*, Monterey, CA, USA, 2002, pp. 16–16.

[22] "Lustre : A scalable, high-performance file system cluster," 2003. [Online]. Available: https://api.semanticscholar.org/CorpusID:16120094

[23] F. Herold and S. Breuner, "An introduction to BeeGFS," 2018. [Online]. Available: https://www.pacificteck.com/wp-content/uploads/pdf/Introduction_to_BeeGFS_by_ThinkParQ.pdf

[24] Y. Lu, Y. Chen, R. Latham, and Y. Zhuang, "Revealing applications' access pattern in collective I/O for cache management," in *Proc. 28th ACM Int. Conf. Supercomput.*, New York, NY, USA, 2014, pp. 181–190.

[25] G. K. Lockwood et al., "A year in the life of a parallel file system," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2018, pp. 931–943.

[26] B. Yang et al., "End-to-end I/O monitoring on a leading supercomputer," in *Proc. 16th USENIX Conf. Networked Syst. Des. Implementation*, 2019, pp. 379–394.

[27] P. Carns et al., "24/7 characterization of petascale I/O workloads," in *Proc. IEEE Int. Conf. Cluster Comput. Workshops*, 2009, pp. 1–10.

[28] C. Wan, J. Sun, M. Snir, K. Mohror, and E. Gonsiorowski, "Recorder 2.0: Efficient parallel I/O tracing and analysis," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. Workshops*, 2020, pp. 1–8.

[29] J. Garlick, "Lustre monitoring tool," 2010. [Online]. Available: https://github.com/LLNL/lmt

[30] M. Isakov et al., "HPC I/O throughput bottleneck analysis with explainable local models," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2020, pp. 1–13.

[31] S. Kim, A. Sim, K. Wu, S. Byna, and Y. Son, "Design and implementation of I/O performance prediction scheme on HPC systems through large-scale log analysis," *J. Big Data*, vol. 10, pp. 1–27, 2023.

[32] J.-W. Park et al., "I/O-signature-based feature analysis and classification of high-performance computing applications," *Cluster Comput.*, vol. 27, pp. 3219–3231, 2023.

[33] M. R. Wyatt et al., "PRIONN: Predicting runtime and IO using neural networks," in *Proc. 47th Int. Conf. Parallel Process.*, 2018, pp. 1–12.

[34] Y. Liu, R. Gunasekaran, X. Ma, and S. S. Vazhkudai, "Server-side log data analytics for I/O workload characterization and coordination on large shared storage systems," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2016, pp. 819–829.

[35] J. Bang et al., "HPC workload characterization using feature selection and clustering," in *Proc. 3rd Int. Workshop Syst. Netw. Telemetry Analytics*, 2020, pp. 33–40.

[36] B. Dong, J. L. Bez, and S. Byna, "Aiio: Using artificial intelligence for job-level and automatic I/O performance bottleneck diagnosis," in *Proc. 32nd Int. Symp. High-Perform. Parallel Distrib. Comput.*, 2023, pp. 155–167.

[37] Y. Liu et al., "Automatic identification of application I/O signatures from noisy server-side traces," in *Proc. 12th USENIX Conf. File Storage Technol.*, 2014, pp. 213–228.

[38] M. Hartung and M. Kluge, "Mapping of raid controller performance data to the job history on large computing systems," in *Proc. Int. Workshop Data Intensive Scalable Comput. Syst.*, 2014, pp. 73–80.

[39] B. Xie et al., "Predicting output performance of a petascale supercomputer," in *Proc. 26th Int. Symp. High-Perform. Parallel Distrib. Comput.*, 2017, pp. 181–192.

[40] K. W. Cameron et al., "MOANA: Modeling and analyzing I/O variability in parallel system experimental design," *IEEE Trans. Parallel Distrib. Syst.*, vol. 30, no. 8, pp. 1843–1856, Aug. 2019.

[41] S. Madireddy et al., "Machine learning based parallel I/O predictive modeling: A case study on lustre file systems," in *Proc. Conf. High Perform. Comput.*, 2018, pp. 184–204.

[42] N. Rajesh, K. Bateman, J. L. Bez, S. Byna, A. Kougkas, and X. -H. Sun, "TunIO: An AI-powered framework for optimizing HPC I/O," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2024, pp. 494–505.

[43] K. Wu, J. Wei, and J. Lin, "Schedp: I/O-aware job scheduling in large-scale production HPC systems," in *Proc. Netw. Parallel Comput.*, 2022, pp. 315–326.

[44] L. Pang and K. Kant, "Server-side workload identification for HPC I/O requests," in *Proc. 2nd Workshop Perform. Eng. Modelling Anal. Vis. Strategy*, New York, NY, USA, 2022, pp. 15–22.

[45] L. Xu et al., "Prediction for distributional outcomes in high-performance computing input/output variability," *J. Roy. Stat. Soc. Ser. C, Appl. Statist.*, vol. 73, no. 3, pp. 561–580, 2024.

[46] A. H. Yazdani, A. K. Paul, A. M. Karimi, F. Wang, and A. Butt, "User-based I/O profiling for leadership scale HPC workloads," in *Proc. 26th Int. Conf. Distrib. Comput. Netw.*, New York, NY, USA, 2025, pp. 181–190.

[47] E. Betke and J. Kunkel, "The importance of temporal behavior when classifying job IO patterns using machine learning techniques," in *Proc. Conf. High Perform. Comput.*, 2020, pp. 191–205.

[48] W. Yang, X. Liao, D. Dong, and J. Yu, "A quantitative study of the spatiotemporal I/O burstiness of HPC application," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2022, pp. 1349–1359.

[49] Y. Li, L. Xiao, J. Feng, J. Zhang, G. Zheng, and Y. Yuan, "Ioscout: An I/O characteristics prediction method for the supercomputer jobs," in *Proc. IEEE 3rd Int. Conf. Comput. Commun. Artif. Intell.*, 2023, pp. 205–210.

[50] J.-W. Park and T. Hong, "Application I/O behavior analysis on leadership cluster system," *J. Supercomput.*, vol. 81, no. 6, 2025, Art. no. 763.

[51] E. Costa et al., "Systematically inferring I/O performance variability by examining repetitive job behavior," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2021, pp. 1–15.

[52] H. Tang et al., "Sctuner: An autotuner addressing dynamic I/O needs on supercomputer I/O subsystems," in *Proc. IEEE/ACM 6th Int. Parallel Data Syst. Workshop*, 2021, pp. 29–34.

[53] B. Yang et al., "End-to-end I/O monitoring on leading supercomputers," *ACM Trans. Storage*, vol. 19, no. 1, pp. 1–35, Jan. 2023.

[54] C. Egersdoerfer, M. H. Rashid, D. Dai, B. Fang, and T. Nathan, "Understanding and predicting cross-application I/O interference in HPC storage systems," in *Proc. Workshops Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2024, pp. 1330–1339.

[55] J. Bang et al., "An in-depth I/O pattern analysis in HPC systems," in *Proc. IEEE 28th Int. Conf. High Perform. Comput. Data Analytics*, 2021, pp. 400–405.

[56] J. Schmid and J. Kunkel, "Predicting I/O performance in HPC using artificial neural networks," *Supercomput. Front. Innov., Int. J.*, vol. 3, no. 3, pp. 19–33, Sep. 2016.

[57] S. Yu, M. Winslett, J. Lee, and X. Ma, "Automatic and portable performance modeling for parallel I/O: A machine-learning approach," *SIGMETRICS Perform*. Eval. Rev., vol. 30, no. 3, pp. 3–5, Dec. 2002.

[58] Y. He, D. Dai, and F. S. Bao, "Modeling HPC storage performance using long short-term memory networks," in *Proc. IEEE 21st Int. Conf. High Perform. Comput. Commun./IEEE 17th Int. Conf. Smart City/IEEE 5th Int. Conf. Data Sci. Syst.*, 2019, pp. 1107–1114.

[59] B. Xie et al., "Applying machine learning to understand write performance of large-scale parallel filesystems," in *Proc. IEEE/ACM 4th Int. Parallel Data Syst. Workshop*, 2019, pp. 30–39.

[60] B. Xie et al., "Interpreting write performance of supercomputer I/O systems with regression models," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2021, pp. 557–566.

[61] L. Xu et al., "Prediction of high-performance computing input/output variability and its application to optimization for system configurations," *Qual. Eng.*, vol. 33, pp. 318–334, 2021.

[62] S. Madireddy et al., "Modeling I/O performance variability using conditional variational autoencoders," in *Proc. 2018 IEEE Int. Conf. Cluster Comput.*, 2018, pp. 109–113.

[63] L. Wan, M. Wolf, F. Wang, J. Y. Choi, G. Ostrouchov, and S. Klasky, "Analysis and modeling of the end-to-end I/O performance on OLCF's Titan supercomputer," in *Proc. IEEE 19th Int. Conf. High Perform. Comput. Commun./IEEE 15th Int. Conf. Smart City/IEEE 3rd Int. Conf. Data Sci. Syst.*, 2017, pp. 1–9.

[64] C. Egersdoerfer et al., "Ion: Navigating the HPC I/O optimization journey using large language models," in *Proc. 16th ACM Workshop Hot Topics Storage File Syst.*, 2024, pp. 86–92.

[65] C. Egersdoerfer, A. Sareen, J. L. Bez, S. Byna, D. D. Xu, and D. Dai, "Ioagent: Democratizing trustworthy HPC I/O performance diagnosis capability via LLMs," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2025, pp. 322–334.

[66] L. Xu et al., "Modeling I/O performance variability in high-performance computing systems using mixture distributions," *J. Parallel Distrib. Comput.*, vol. 139, pp. 87–98, 2020.

[67] H. Ather, J. L. Bez, B. Norris, and S. Byna, "Illuminating the I/O optimization path of scientific applications," in *Proc. 38th Int. Conf. High Perform. Comput.*, Berlin, Heidelberg, Springer-Verlag, 2023, pp. 22–41.

[68] J. M. Kunkel, M. Zimmer, and E. Betke, "Predicting performance of non-contiguous I/O with machine learning," in *Proc. Inf. Secur. Conf.*, 2015, pp. 257–273.

[69] M. H. Rashid, X. Li, Y. He, F. S. Bao, and D. Dai, "Dial: Decentralized I/O autotuning via learned client-side local metrics for parallel file system," in *Proc. IEEE 25th Int. Symp. Cluster*, Cloud and Internet Comput., Troms, Norway, 2025, pp. 1–4.

[70] S. Kumar et al., "Characterization and modeling of pidx parallel I/O for performance optimization," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2013, pp. 1–12.

[71] A. J. S. Tipu, P. O. Conbhui, and E. Howley, "Artificial neural networks based predictions towards the auto-tuning and optimization of parallel IO bandwidth in HPC system," *Cluster Comput.*, vol. 27, no. 1, pp. 71–90, Dec. 2022.

[72] B. Behzad, S. Byna, S. M. Wild Prabhat, and M. Snir, "Dynamic model-driven parallel I/O performance tuning," in *Proc. IEEE Int. Conf. Cluster Comput.*, 2015, pp. 184–193.

[73] F. Isaila et al., "Collective I/O tuning using analytical and machine learning models," in *Proc. IEEE Int. Conf. Cluster Comput.*, 2015, pp. 128–137.

[74] M. Agarwal, P. Jain, D. Singhvi, and P. Malaka, "Execution- and prediction-based auto-tuning of parallel read and write parameters," in *Proc. IEEE 23rd Int. Conf. High Perform. Comput. Commun./7th Int. Conf. Data Sci. Syst./19th Int. Conf. Smart City/7th Int. Conf. Dependability Sensor Cloud Big Data Syst. Appl.*, 2021, pp. 587–594.

[75] M. Dorier et al., "HPC storage service autotuning using variational-autoencoder -guided asynchronous bayesian optimization," in *Proc. 2022 IEEE Int. Conf. Cluster Comput.*, 2022, pp. 381–393.

[76] Z. Liu et al., "Optimizing HPC I/O performance with regression analysis and ensemble learning," in *Proc. IEEE Int. Conf. Cluster Comput.*, 2023, pp. 234–246.

[77] Y. Li et al., "Capes: Unsupervised storage performance tuning using neural network-based deep reinforcement learning," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2017, pp. 1–14.

[78] Z. Wentao, W. Lu, and C. Yaodong, "Performance optimization of lustre file system based on reinforcement learning," *J. Comput. Res. Develop.*, vol. 56, 2019, Art. no. 1578.

[79] H. Zhu, D. Scheinert, L. Thamsen, K. Gontarska, and O. Kao, "Magpie: Automatically tuning static parameters for distributed file systems using deep reinforcement learning," in *Proc. 2022 IEEE Int. Conf. Cloud Eng.*, 2022, pp. 150–159.

[80] K. Lu et al., "Adsts: Automatic distributed storage tuning system using deep reinforcement learning," in *Proc. 51st Int. Conf. Parallel Process.*, 2023, pp. 1–13.

[81] Z. Tian, C. Yu, Z. Shan, S. Yin, H. Xu, and B. Zhao, "Experience-based parallel ddpg for automatic tuning of ceph knobs," in *Proc. 3rd Int. Conf. Cloud Comput. Big Data Appl. Softw. Eng.*, 2024, pp. 780–786.

[82] J. Zhou, Y. Chen, D. Dai, Y. Zhuang, and W. Wang, "I/O characteristic discovery for storage system optimizations," *J. Parallel Distrib. Comput.*, vol. 148, pp. 1–13, 2021.

[83] J. Oly and D.A. Reed, "Markov model prediction of I/O requests for scientific applications," in *Proc. 16th Int. Conf. Supercomputing*, Association for Computing Machinery,, 2002, pp. 147–155.

[84] T. Madhyastha and D. Reed, "Learning to classify parallel input/output access patterns," *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 8, pp. 802–813, Aug. 2002.

[85] P. Subedi et al., "Stacker: An autonomic data movement engine for extreme-scale data staging-based in-situ workflows," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2018, pp. 920–930.

[86] P. Subedi, P. E. Davis, and M. Parashar, "Leveraging machine learning for anticipatory data delivery in extreme scale in-situ workflows," in *Proc. 2019 IEEE Int. Conf. Cluster Comput.*, 2019, pp. 1–11.

[87] L. Wan, Z. Lu, Q. Cao, F. Wang, S. Oral, and B. Settlemyer, "Ssd-optimized workload placement with adaptive learning and classification in HPC environments," in *Proc. 30th Symp. Mass Storage Syst. Technol.*, 2014, pp. 1–6.

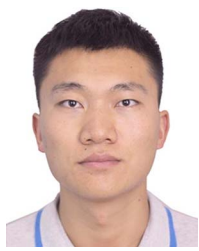
[88] P. Cheng et al., "Optimizing data placement on hierarchical storage architecture via machine learning," in *Proc. Conf. Netw. Parallel Comput.*, 2019, pp. 289–302.

[89] Y. Xu, P. Sivaraman, H. Devarajan, K. Mohror, and A. Bhatele, "Ml-based modeling to predict I/O performance on different storage sub-systems," in *Proc. IEEE 31st Int. Conf. High Perform. Comput. Data Analytics*, 2024, pp. 221–231.

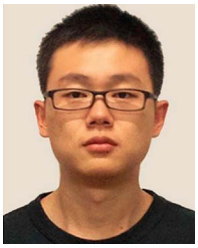
[90] W. Wang, H. Wu, L. Yang, Z. Liu, and B. Zheng, "Aio: Automating I/O optimization pipeline for data-intensive applications in HPC," in *Proc. 2024 IEEE Int. Symp. Parallel Distrib. Process. Appl.*, 2024, pp. 1541–1548.
[91] F. Monjalet and T. Leibovici, "Predicting file lifetimes with machine learning," in *Proc. Int. Workshops High Perform. Comput.*, Frankfurt, Germany, Springer-Verlag, 2019, pp. 288–299.
[92] L. Thomas et al., "Predicting file lifetimes for data placement in multi-tiered storage systems for HPC," *SIGOPS Oper. Syst. Rev.*, vol. 55, no. 1, pp. 99–107, Jun. 2021.
[93] L.-M. Nicolas et al., "Slrl: A simple least remaining lifetime file eviction policy for HPC multi-tier storage systems," *SIGOPS Oper. Syst. Rev.*, vol. 56, no. 1, pp. 70–76, Jun. 2022.
[94] A. Khelili, S. R. Hayek, and S. Zertal, "Forecasting file lifecycles for intelligent data placement in hierarchical storage," in *Proc. IEEE 35th Int. Symp. Comput. Archit. High Perform. Comput.*, 2023, pp. 181–191.
[95] A. Khelili, S. R. Hayek, and S. Zertal, "Pattern matching to improve accuracy and efficiency of file lifecycles forecasting," in *Proc. 14th Int. Conf. Intell. Systems: Theories Appl.*, 2023, pp. 1–8.
[96] S. Karki, B. Nguyen, J. Feener, K. Davis, and X. Zhang, "Enforcing end-to-end I/O policies for scientific workflows using software-defined storage resource enclaves," *IEEE Trans. Multi-Scale Comput. Syst.*, vol. 4, no. 4, pp. 662–675, Oct.–Dec. 2018.
[97] W. Shi, P. Cheng, C. Zhu, and Z. Chen, "An intelligent data placement strategy for hierarchical storage systems," in *Proc. IEEE 6th Int. Conf. Comput. Commun.*, 2020, pp. 2023–2027.
[98] W. Yang, C. Chen, and J. Yu, "Temporal staggering of applications based on job classification and I/O burst prediction," in *Proc. IEEE 24th Int. Conf. High Perform. Comput. Commun./8th Int. Conf. Data Sci. Syst./20th Int. Conf. Smart City/8th Int. Conf. Dependability Sensor Cloud Big Data Syst. Appl.*, 2022, pp. 965–970.
[99] B. Zha and H. Shen, "Probabilistic scheduling of dynamic I/O requests via application clustering for burst-buffers equipped high-performance computing," *Concurrency Comput. Pract. Experience*, vol. 36, 2024, Art. no. e8142.
[100] K. Bateman et al., "Luxio: Intelligent resource provisioning and auto-configuration for storage services," in *Proc. IEEE 29th Int. Conf. High Perform. Comput. Data, Analytics*, 2022, pp. 246–255.
[101] M. R. Wyatt et al., "Ai4io: A suite of ai-based tools for IO-aware scheduling," *Int. J. High Perform. Comput. Appl.*, vol. 36, no. 3, pp. 370–387, May 2022.
[102] M. R. Wyatt, S. Herbein, K. Shoga, T. Gamblin, and M. Taufer, "CanarIO: Sounding the alarm on IO-related performance degradation," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2020, pp. 73–83.
[103] E. Saeedizade, R. Taheri, and E. Arslan, "I/O burst prediction for HPC clusters using darshan logs," in *Proc. IEEE 19th Int. Conf. e-Sci.*, 2023, pp. 1–10.
[104] L. Wan et al., "I/O performance characterization and prediction through machine learning on HPC systems," 2020. [Online]. Available: https://cug.org/proceedings/cug2020_proceedings/includes/files/spec102s1.pdf
[105] A. K. Paul et al., "I/O load balancing for Big Data HPC applications," in *Proc. IEEE Int. Conf. Big Data*, 2017, pp. 233–242.
[106] P. Subedi, P. E. Davis, and M. Parashar, "RISE: Reducing I/O contention in staging-based extreme-scale in-situ workflows," in *Proc. IEEE Int. Conf. Cluster Comput.*, 2021, pp. 146–156.
[107] G. Xian et al., "Mobilizing underutilized storage nodes via job path: A job-aware file striping approach," *Parallel Comput.*, vol. 121, 2024, Art. no. 103095.
[108] B. Yang, Y. Zou, W. Liu, and W. Xue, "An end-to-end and adaptive I/O optimization tool for modern HPC storage systems," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2022, pp. 1294–1304.
[109] J. L. Bez, F. Z. Boito, R. Nou, A. Miranda, T. Cortes, and P. O. A. Navaux, "Detecting I/O access patterns of HPC workloads at runtime," in *Proc. 31st Int. Symp. Comput. Archit. High Perform. Comput.*, 2019, pp. 80–87.
[110] F. Z. Boito et al., "Automatic I/O scheduling algorithm selection for parallel file systems," *Concurrency Comput. Pract. Experience*, vol. 28, pp. 2457–2472, 2016.
[111] X. Zhang, K. Davis, and S. Jiang, "QoS support for end users of I/O-intensive applications using shared storage systems," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, New York, NY, USA, 2011, pp. 1–12.
[112] J. L. Bez et al., "Adaptive request scheduling for the I/O forwarding layer using reinforcement learning," *Future Gener. Comput. Syst.*, vol. 112, pp. 1156–1169, 2020.
[113] W. Cheng et al., "AIOC^2: A deep Q-learning approach to autonomic I/O congestion control in lustre," *Parallel Comput.*, vol. 108, 2021, Art. no. 102855.
[114] A. K. Paul et al., "Machine learning assisted HPC workload trace generation for leadership scale storage systems," in *Proc. 31st Int. Symp. High-Perform. Parallel Distrib. Comput.*, 2022, pp. 199–212.
[115] N. Moti et al., "The I/O trace initiative: Building a collaborative I/O archive to advance HPC," in *Proc. Workshops Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2023, pp. 1216–1222.
[116] D. Povaliaiev, R. Liem, J. Kunkel, J. Lofstead, and P. Carns, "High-quality I/O bandwidth prediction with minimal data via transfer learning workflow," in *Proc. IEEE 36th Int. Symp. Comput. Archit. High Perform. Comput.*, 2024, pp. 93–104.
[117] M. Isakov et al., "A taxonomy of error sources in HPC I/O machine learning models," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2022, pp. 1–14.
[118] S. Madireddy et al., "Adaptive learning for concept drift in application performance modeling," in *Proc. 48th ACM Int. Conf. Parallel Process.*, 2019, pp. 1–11.
[119] M. Isakov et al., "Toward generalizable models of I/O throughput," in *Proc. 2020 IEEE/ACM Int. Workshop Runtime Operating Syst. Supercomputers*, 2020, pp. 41–49.
[120] A. Giannakou, D. Hazen, B. Enders, L. Ramakrishnan, and N. J. Wright, "Understanding data movement patterns in HPC: A nersc case study," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2024, pp. 1–17.
[121] D. Biswas, S. Neuwirth, A. K. Paul, and A. R. Butt, "Bridging network and parallel I/O research for improving data-intensive distributed applications," in *Proc. IEEE Workshop Innovating Netw. Data-Intensive Sci.*, 2021, pp. 50–56.
[122] M. Mubarak et al., "Quantifying I/O and communication traffic interference on dragonfly networks equipped with burst buffers," in *Proc. IEEE Int. Conf. Cluster Comput.*, 2017, pp. 204–215.

Jingxian Peng received the BS degree from Chongqing University in 2023. She is currently working toward the master's degree with the College of Computer Science and Technology, National University of Defense Technology. Her research interests include high performance computing and resource management.

Lihua Yang received the BE and PhD degrees in computer science and technology from the Huazhong University of Science and Technology, in 2016 and 2022, respectively. She is currently a postdoctoral fellow with the College of Computer Science and Technology, National University of Defense Technology. Her research interests include computer system architecture, file systems, and high performance computing.



Huijun Wu received the BE and ME degrees from the National University of Defense Technology in 2013 and 2015, respectively. He received the PhD degree in computer science and engineering from the University of New South Wales, Australia, in 2019. He is currently an assistant professor with the College of Computer Science and Technology, National University of Defense Technology. His research interests include high performance computing, file systems, and graph neural network.



Wenzhe Zhang was born in 1988. He received the BS, MS, and PhD degrees from the College of Computer Science and Technology, National University of Defense Technology in 2010, 2012, and 2017, respectively. He is currently an associate professor with the College of Computer Science and Technology, National University of Defense Technology. His research interests include high-performance computing and operating system.

Zhenwei Wu received the BE degree from Tsinghua University in 2014. He received the master's and PhD degrees in computer science and technology from the National University of Defense Technology, in 2016 and 2020, respectively. He is currently an assistant professor with the College of Computer Science and Technology, National University of Defense Technology. His research interests include high performance computing and operating system.

Wei Zhang received the PhD degree from the National University of Defense Technology in 2010. He is currently an associate professor with the College of Computer Science and Technology, National University of Defense Technology. His research interests include high performance computing, parallel file system, MPI, and high speed interconnection network.

Jiaxin Li received the master's and PhD degrees in computer science and technology from the National University of Defense Technology, in 2013 and 2018, respectively. He is currently an assistant professor with the College of Computer Science and Technology, National University of Defense Technology. His research interests include operating systems technology, distributed computing, and high-performance computing.

Yiqin Dai received the BS degree from the National University of Defense Technology in 2019. He is currently working toward the PhD degree with the College of Computer Science and Technology, National University of Defense Technology. His research interests include high performance computing and resource management.

Yong Dong received the PhD degree from the National University of Defense Technology in 2012. He is currently a professor with the College of Computer Science and Technology, National University of Defense Technology. His research interests include high performance computing, parallel storage, MPI, and resource management.